Do Generate–Verify–Repair Guards Improve LLM NetOps Outputs? A Paired Emulator Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a paired confirmatory experiment (n=200 natural-language NetOps intents; study_id cnd-001-5f3a) that compares ungated LLM generation to a bounded generate–verify–repair (GVR) pipeline applied to a single shared LLM candidate per intent. Generation used gpt-5-mini and the guarded arm applied a deterministic three-stage validator and a deterministic, rule-based repair operator with repair_budget <=1 (no extra LLM calls). Artifact-backed primary outcomes: baseline successes = 199/200 (0.995), guarded = 200/200 (1.000); paired contingency n11=199, n10=1, n01=0, n00=0; paired difference (guarded - baseline) = 0.005. The preregistered exact two-sided McNemar (exact binomial on discordant pairs) yields p = 1.0; paired-bootstrap (10,000 resamples, seed 1729) 95% percentile CI = [0.0, 0.015]. The preregistered 0.15 absolute-effect target is excluded by the observed CI because the realized baseline was near-ceiling. We provide concise, auditable descriptions of the validator and repair operator, execution accounting (model_calls_used = 201; deterministic repair invocations = 1), exact-test justification appropriate for small discordant counts, and operationally grounded limitations. All prompts, provider traces, validator traces, repair traces (the single invoked repair trace), and analysis artifacts are archived in the study evidence bundle.**

I. Introduction

Automating routine network-operations (NetOps) changes with large language models (LLMs) promises reduced manual effort but raises an operational-safety question: do LLM-produced configuration artifacts execute correctly when applied to control planes? Prior work demonstrates intent-to-configuration prototypes and documents structured-output failures (missing commands, misordering, protected-field modification) and proposes mitigations such as retrieval-augmentation, constrained decoding, verifier-assisted repair, and multi-stage tool-augmentation. Despite these proposals, there is limited execution-grounded evidence that quantifies the marginal benefit of applying a bounded deterministic repair to the same initial LLM candidate (a low-hosted-call operational estimand).

We preregistered and executed a paired experiment (study_id cnd-001-5f3a) that isolates the "repair-on-same-candidate" question by sharing a single initial LLM candidate across both arms (baseline ungated acceptance and guarded generate–verify–repair). The shared-candidate estimand is operationally relevant when hosted-model call budgets are constrained. Our contributions are: (1) a preregistered, artifact-backed paired evaluation under the shared-candidate estimand; (2) concise algorithmic descriptions of the deterministic three-stage validator and the deterministic, rule-based repair operator used in the guarded pipeline; and (3) complete execution accounting and exact-test justification for small-discordant-pair inference.

II. Related Work

Research on LLMs for NetOps has rapidly diversified along three complementary directions: intent-to-configuration translation and co-pilot prototypes, structured-output mitigation techniques, and formal/deterministic validators that can be integrated into tool-augmented pipelines. Prior prototype and survey work document practical promise—natural-language intents can be translated into device-level commands and assist operators in configuration generation—but also recurrent failure modes such as missing required commands, incorrect ordering, and unintended modification of protected fields (intent-translation prototypes summarized in recent literature) [doi:10.1002/nem.2313; https://openalex.org/W7161165783].

Mitigation efforts borrow from nearby domains. Retrieval-augmented generation (RAG), live schema grounding, and constrained decoding have shown material reductions in hallucinated or structurally invalid outputs in NL2SQL and document-grounded tasks; these approaches are natural candidates for NetOps but require careful adaptation to device CLI dialects, workflow policies, and operational constraints [doi:10.2139/ssrn.6909831]. Similarly, constrained or grammar-aware decoding methods and deterministic post-hoc canonicalizers reduce syntactic errors but do not by themselves guarantee correct control-plane behavior when applied to live or emulated networks.

Deterministic verification and formal-methods work (NetKAT variants, weighted NetKAT and ensemble verifiers) provide a complementary assurance axis by checking reachability, isolation, and quantitative properties of candidate configurations; these techniques can produce machine-readable violation codes that are actionable for automated repair or operator review [doi:10.1145/3808318]. However, verification tools face two practical constraints when applied in NetOps pipelines: (1) fidelity to dynamic control-plane phenomena (timing, convergence, vendor-specific idiosyncrasies) and (2) scalability when reasoning over larger multi-device changes. These limits motivate hybrid architectures that combine fast deterministic checks with auditable traces and bounded repair operations.

Security- and safety-oriented evaluations emphasize adversarial risks: human-readable jailbreak prompts and adversarial-context attacks remain an effective threat class for unconstrained generative pipelines and therefore constitute an op-

erational risk for NetOps automation unless mitigations and guards are deployed [doi:10.1145/3815174]. Architectural discussions of agentic NetOps emphasize that production reliability depends on the surrounding assurance machinery—auditable evidence traces, bounded tool use, rollback policies, and independent validators—rather than on raw model capability alone (see recent agentic NetOps position work) [https://openalex.org/W7161165783].

Across these literatures the recurring gap is empirical and execution-grounded: many studies show reduced failure modes in isolated subcomponents (generation, decoding, or verification) but there are relatively few reproducible, artifact-backed measurements of end-to-end generate–validate–repair pipelines under practical constraints such as limited hosted-model calls, deterministic repair budgets, and auditable validator traces. Our study targets this specific gap by measuring the marginal effect of a bounded deterministic repair applied to the same initial LLM candidate under an explicit shared-generation budget; this positions our work as an execution-grounded complement to prior prototype, mitigation, and verification literature [doi:10.1002/nem.2313; doi:10.2139/ssrn.6909831; doi:10.1145/3808318; https://openalex.org/W7161165783; doi:10.1145/3815174].

III. Methodology

Preregistration and overall design: We preregistered a paired, shared-initial-generation confirmatory protocol (study_id cnd-001-5f3a). The confirmatory set contains 200 curated natural-language intents covering single-device and small multi-device changes (VLAN/MTU edits, uplink switchover, safe shutdown-configure-restore patterns). For each intent the hosted model produced a single initial candidate (shared across arms). The baseline arm accepts that candidate verbatim; the guarded arm applies a deterministic three-stage validator and, if the validator reports a repairable violation family, invokes at most one deterministic repair transformation (repair_budget <=1) implemented in code (no additional LLM calls). The primary estimand is the paired_success_rate_difference_guarded_minus_baseline and the preregistered primary hypothesis test is McNemar's paired binary test implemented in its exact-binomial form (see Statistical methods below). Planned maximum hosted-model calls = 400; realized model_calls_used = 201 (shared_initial_generation = 200; repair-stage calls = 1). The execution environment used Dockerized Mininet + FRR images and deterministic_netops_validator_v5 as the validator; all run artifacts are archived in the evidence bundle.

Task bank, canonicalization, and scoring: Each confirmatory intent is paired with an explicit expected_final_state and a required_sequence of commands encoded in the task manifest. Scoring follows the preregistered full_intent_constraint_validation rule and requires: (a) syntactic parse and canonical normalization of CLI-like candidate lines into tokenized command records; (b) presence of every command in the task's required_sequence (MISSING_REQUIRED_COMMAND flagged otherwise); (c) ordering/dependency checks (e.g., make-before-break, shutdown_configure_restore) producing dependency violation codes when ordering constraints are unmet; (d) group-activity and minimum-active constraints (e.g., final-group constraints for uplink pairs); and (e) preservation/protection checks for interfaces marked as protected. After canonicalization the deterministic emulator applies the normalized_configuration and computes observed final_state; per-interface, per-field equality with expected_final_state is required for a passing score. The validator emits a machine-readable trace per episode containing fields such as parsed_command_count, observed_touched_field_count, normalized_configuration, violation_codes (families and deterministic subcodes), validation_after.valid, and validator_version. These traces are the inputs for repair decisions and are archived for audit.

Deterministic validator (three-stage implementation semantics): The validator executes these stages deterministically in order: (1) Syntactic parse and normalization: line-oriented parsing into a canonical command tuple (interface, field, value), removal of insignificant whitespace and comments, and canonical ordering of intra-interface commands when unambiguous; parse failures produce SYNTAX_PARSE_ERROR and are unrecoverable by deterministic repair. (2) Structural and workflow checks: this stage compares parsed commands to the task payload (required_sequence and workflow_policies), verifies ordering constraints and group-activity invariants, and emits deterministic violation families (examples observed in traces: MISSING_REQUIRED_COMMAND, DEPENDENCY_VIOLATION, PROTECTED_INTERFACE_MODIFICATION). (3) Deterministic emulator application: the normalized_configuration is applied to a deterministic control-plane emulator that updates per-interface fields; the resulting observed final_state is compared to expected_final_state using exact field equality checks for the set of fields declared in the task (admin, mtu, vlan). The validator records counts of parsed and required commands and a final boolean valid flag. All emitted traces are used both for scoring and for deterministic repair logic.

Deterministic repair operator: design constraints and decision rule - Design constraints (preregistered and enforced): repairs must be implemented by deterministic code only (no extra LLM calls); at most one repair per episode for the confirmatory run; repairs are applied only when the validator reports a violation family within a preregistered repairable set and when the mapping from violation to deterministic transformation is unique and unambiguous. - Repairable violation families (operationalized from the task manifest and validator schema): MISSING_REQUIRED_COMMAND and DEPENDENCY_VIOLATION are considered repairable by insertion or reordering; PROTECTED_INTERFACE_MODIFICATION is handled by deterministic replacement/redaction to the protected value. Violations involving ambiguous intent interpretation, syntactic parse failures, or multiple plausible insertion sites are treated as nonrepairable and left unchanged. -

Deterministic transformations implemented (canonical rules): 1) Insertion: when a required command is missing and the task manifest supplies an explicit required_sequence containing that command, insert the missing command at a canonical insertion point: immediately after the last preceding required_sequence command if present; otherwise at the canonical position defined by the task's command grouping (interface-ordered canonical slot). The inserted command text is constructed deterministically from the task manifest required_sequence and normalized to the validator's canonical form. 2) Reordering: when the validator reports DEPENDENCY_VIOLATION where a known dependency order is provided in the task payload, atomically reorder contiguous command blocks by interface into the required_sequence order while preserving group boundaries; reordering is deterministic and performed only if a single deterministic reorder resolves the violation. 3) Preservation enforcement (redaction/replacement): when a candidate modifies a protected interface or an unspecified state that must be preserved, deterministically replace the offending command(s) with the canonical preserved value taken from the task's initial_state entry. - Repair decision predicate: apply a repair iff (violation_family $\in$ repairable_set) AND (deterministic_mapping_is_unique == true) AND (repair_budget_remaining > 0). After applying the single deterministic transformation, re-run the validator stages to compute validation_after and final_state; the result is archived in a repair trace.

Repair instrumentation and measured behavior: The preregistered run recorded deterministic_repair_invocations = 1 and that single invocation converted the sole validator-failing candidate into a validator-passing normalized_configuration, producing the lone guarded-only improvement (n10 = 1). Across the confirmatory set syntactic/parse failures = 0. Provider-call accounting (artifact-backed) shows hosted_model_call_event_count = 201 (shared_initial_generation = 200; repair-stage calls = 1), completed_hosted_model_call_event_count = 201, and cache_miss_count = 201.

Statistical methods (exact paired inference and bootstrap uncertainty): The primary confirmatory test is McNemar's paired test implemented via its exact-binomial formulation appropriate when discordant-pair counts are small. Let n = n10 + n01 be the number of discordant pairs and k = n10 the count favoring the guarded arm. Under H0 (no difference) each discordant pair has probability 0.5 of falling in either order, so the two-sided p-value is computed from the binomial distribution Binomial(n, 0.5) by doubling the smaller tail probability (exact two-sided binomial test on k successes). For our observed discordant counts (n10 = 1, n01 = 0 => n = 1, k = 1) the exact two-sided p-value equals 1.0 (artifact-backed p reported in analysis results). Complementary uncertainty quantification uses a paired-bootstrap percentile 95% CI on the paired difference (guarded - baseline) with 10,000 resamples (bootstrap_seed = 1729), both preregistered and archived. The preregistered power analysis targeted an absolute effect of 0.15 at n = 200 under a baseline assumption $\approx$0.5; because the realized baseline was near-ceiling (0.995), attainable discordant counts were compressed and the test's sensitivity to large absolute effects was reduced; this consequence is anticipated by the preregistration and is described in the limitations.

Missingness, retries, and contamination controls: Provider/API generation failures were predeclared as failures for the affected arm; none occurred in the confirmatory run. The preregistered retry policy allowed a single automatic retry for transient network/API timeouts; all retries and provider-call audit records are archived. Contamination detection against pre-lock artifacts and near-duplicates is part of the manifest ingestion and was applied to exclude any contaminated tasks from confirmatory analysis; no confirmatory episodes were excluded. Full per-episode task payloads (including declared difficulty labels and required_sequence entries) are recorded in the archived execution/task_manifest.jsonl for audit and reanalysis.

Reproducibility and artifact pointers (concise): all inputs and outputs required to reproduce the validation and repair decisions are archived: per-episode normalized_configurations, validator traces, repair trace(s), model provider traces, execution logs, analysis scripts, and the preregistration manifest. The model configuration used for generation declares the requested model (gpt-5-mini) and execution parameters; the evidence bundle includes the execution manifest and analysis artifacts referenced in this paper for independent inspection and reanalysis.

IV. Results

Execution and primary outcomes: The confirmatory run completed all planned episodes (400 episodes = 200 pairs) with no provider/API failures. Artifact-backed primary counts: baseline_success_count = 199/200 (0.995) and guarded_success_count = 200/200 (1.0). The paired contingency table is n11 = 199, n10 = 1 (guarded-only), n01 = 0 (baseline-only), n00 = 0; paired difference (guarded - baseline) = 0.005. The preregistered exact two-sided McNemar (exact binomial on discordant pairs) yields p = 1.0 for the observed ordering (n = 1). The paired-bootstrap (10,000 resamples, seed 1729) 95% percentile CI is [0.0, 0.015]. All per-episode JSONL scoring artifacts and the analysis result JSON are archived.

Repair and failure accounting: validation_before.valid == false occurred in exactly one confirmatory episode; deterministic repair invocations = 1 and that single repair converted the failing candidate into a validator-passing final configuration, producing the sole guarded-only success (n10 = 1). Syntactic/parse failures across the confirmatory set = 0. Provider-call audit: hosted_model_call_event_count = 201, completed_hosted_model_call_event_count = 201, cache_miss_count = 201, stage_counts show shared_initial_generation = 200 and repair = 1. No episodes were excluded for contamination; missingness summary reports complete_pairs = 200 and zero unscorable episodes.

Representative exemplars and difficulty context: we archive six representative exemplars with per-example difficulty la-

bels (medium = 1, hard = 2, very_hard = 3) to illustrate typical intent patterns including shutdown_configure_restore, make-before-break uplink switchover, and paired atomic MTU changes. The full per-episode difficulty distribution for all 200 confirmatory intents is recorded in the archived task manifest (execution/task_manifest.jsonl) and is available for re-analysis; because aggregated counts are stored in that manifest we reference it for exhaustive difficulty counts rather than enumerating the full distribution inline.

Sensitivity and interpretive statistics: The preregistered 0.15 absolute-effect target was chosen under a baseline $\approx$0.5 assumption and is excluded by the observed bootstrap CI; the near-ceiling baseline compresses attainable discordant counts and reduces power to detect large absolute improvements under the shared-candidate estimand. Practically, deterministic repair-on-same-candidate produced a single marginal improvement in this curated confirmatory bank.

V. Discussion

Operational interpretation: Under the shared-initial-generation estimand and the curated confirmatory distribution the ungated gpt-5-mini generator produced near-perfect candidates (199/200). Deterministic, auditable validators produce structured violation codes that enable targeted deterministic repairs; in low-error regimes the marginal yield of a conservative rule-based repair on the same candidate is small but auditable and low-cost. For operators constrained by model-call budgets, the guarded shared-candidate design preserves low hosted-call cost while providing an assurance gate; for contexts prioritizing maximal success, independent resampling or LLM-invoked repair loops merit exploration.

Design-space trade-offs and practitioner guidance: The confirmatory run's execution accounting (model_calls_used = 201; repair invocations = 1; completed_episodes = 400) enables simple cost-versus-yield comparisons. If operators accept higher hosted-call expense, independent-resampling (two or more independent LLM generations per intent) or LLM-guided iterative repair could raise success rates at the cost of greater provider usage and more complex governance. Deterministic validators should be considered mandatory assurance machinery where auditable, repeatable checks and low-latency failure explanations are required.

Threats to validity: Internal validity is strong for the paired estimand and deterministic scoring rules; construct validity depends on emulator fidelity—our deterministic emulator models control-plane state deterministically but cannot fully capture runtime effects such as BGP convergence timing or specific device firmware idiosyncrasies. External validity is limited by the single LLM (gpt-5-mini) and a synthetic curated bank focused on small changes; extrapolation to multivendor CLI diversity or adversarial/out-of-distribution intents is limited. Conclusion validity is constrained by the near-ceiling baseline that reduced discordant-pair counts; the exact McNemar is the correct small-sample test for this observed pattern but offers limited sensitivity when n is small. These limits are documented in the preregistration and archived manifests.

Reproducibility and auditability: All artifacts needed to reproduce analysis and inspection are archived in the evidence bundle: per-episode prompts and responses, provider-call traces, validator traces, deterministic repair trace (the single invoked repair), task manifest entries, analysis scripts, paired contingency CSV, bootstrap parameters (seed = 1729; resamples = 10,000), and execution manifest (preregistration_sha256: e5c5fdbd717c...). The exact-test implementation is the exact-binomial McNemar formulation applied to discordant-pair counts (n10,n01) and is described above; the analysis log records the p-value and bootstrap CI used in the manuscript.

VI. Limitations

- Synthetic, curated confirmatory task bank focused on small single- and multi-device changes; not comprehensive of production NetOps workloads (multivendor CLI dialects, hardware idiosyncrasies, dynamic control-plane timing).
- Single LLM (gpt-5-mini) evaluated; findings do not imply multi-model generality.
- Deterministic emulator + validator model control-plane behavior but cannot fully capture real-world runtime dynamics such as BGP convergence timing or vendor-specific runtime effects.
- Preregistered repair budget limited to a single deterministic repair iteration; different repair strategies or additional iterations might yield different outcomes.
- Shared-initial-generation design reduces model-call cost and isolates repair-on-same-candidate effectiveness but reduces external generalizability compared with independent-resampling estimands.
- Near-ceiling baseline success limited statistical power to analyze failure-mode distributions in the confirmatory set; exploratory failure-taxonomy conclusions are therefore constrained.
- Adversarial or out-of-distribution intents (e.g., jailbreaks, malformed ambiguous intents) were not included in the confirmatory set and require separate evaluation.
- Full per-task difficulty label counts for all 200 confirmatory intents are recorded in execution/task_manifest.jsonl in the archived evidence bundle; the manuscript references that manifest rather than enumerating the full 200-line distribution inline to preserve concise presentation while preserving auditable reproducibility.

VII. Conclusion

We executed an autonomy-locked, preregistered paired experiment to measure whether a bounded generate–verify–repair guard improves emulator-executable success for LLM-generated NetOps configurations under a shared-candidate generation budget. Ungated gpt-5-mini generation succeeded in 199/200 confirmatory cases; the deterministic guarded pipeline succeeded in 200/200. The observed paired difference = 0.005 (exact McNemar two-sided p = 1.0; paired-bootstrap 95% CI [0.0, 0.015]) does not support the preregistered 0.15

absolute-effect target because the baseline was near-ceiling. For this estimand and task distribution, deterministic repair-on-same-candidate yielded negligible marginal improvement, but deterministic validators remain valuable, auditable assurance gates for operational NetOps pipelines. Full artifacts and analysis code are archived with the study for independent audit and re-analysis.

Disclosure Statement

This study was executed autonomously after an autonomy lock defined by the initial master prompt archived at provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d9 6940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human involvement prior to the lock was limited to selecting the topic family (Generative AI and LLMs for NetOps) and approving the master prompt. No manual human editing of technical content, figures, captions, references, results, or manuscript text occurred after the lock. The autonomous pipeline performed literature search and verification, research-gap selection, preregistration (study_id cnd-001-5f3a), code generation, experiment execution, artifact capture, analysis, internal critique, peer-review checks, and manuscript drafting.

Models and tooling: generation requested gpt-5-mini (declared_model_version in execution/model_configuration.json). Validation used deterministic_netops_validator_v5 implementing syntactic parsing, structural/workflow checks, and deterministic emulator application. Repairs in the confirmatory run were applied by deterministic code only (no additional LLM calls). The execution environment used Dockerized Mininet+FRR images orchestrated by adapter family hosted_netops_gvr_v1. Preregistered and enforced settings (not altered post-lock) include: primary estimand = paired_success_rate_difference_guarded_minus_baseline; confirmatory sample = n=200 paired intents; maximum model-call budget = 400; repair budget <=1 deterministic repair iteration; primary analysis = exact McNemar two-sided (exact-binomial on discordant pairs); bootstrap CIs = 10,000 resamples, seed = 1729. Execution accounting (artifact-backed): the run completed 400 episodes with model_calls_used = 201 (shared_initial_generation = 200; repair-stage calls = 1). All prompts, provider-call traces, responses, validator traces, repair traces (the single invoked repair trace), analysis artifacts, and the execution manifest are archived in the study evidence bundle.

References

[1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313

[2] Emmanuel Suárez Acevedo, Tiago Ferreira, Kevin Batz, Oliver Bøving, Nate Foster, Alexandra Silva, "Weighted NetKAT: A Programming Language for Quantitative Network Verification", 2026, 10.1145/3808318

[3] Sanjay Mishra, Divya Chukkapalli, Ganesh R. Naik, "Schema-Aware Localisation (SAL): Live Schema Grounding and Hallucination Validation for Oracle NL2SQL", 2026, 10.2139/ssrn.6909831

[4] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729

[5] Nilanjana Das, Edward Raff, Aman Chadha, Manas Gaur, "Human-Readable Adversarial Prompts: An Investigation into LLM Vulnerabilities Using Situational Context", 2026, 10.1145/3815174